



A Constructive Model of Directed Univalence in Bicubical Sets

Matthew Z. Weaver

Department of Computer Science
Princeton University
mzw@cs.princeton.edu

Daniel R. Licata

Department of Mathematics and Computer Science
Wesleyan University
dlicata@wesleyan.edu

Abstract

Directed type theory is an analogue of homotopy type theory where types represent categories, generalizing groupoids. A bisimplicial approach to directed type theory, developed by Riehl and Shulman, is based on equipping each type with both a notion of path and a separate notion of directed morphism. In this setting, a directed analogue of the univalence axiom asserts that there is a universe of covariant discrete fibrations whose directed morphisms correspond to functions—a higher-categorical analogue of the category of sets and functions. In this paper, we give a constructive model of a directed type theory with directed univalence in bicubical, rather than bisimplicial, sets. We formalize much of this model using Agda as the internal language of a 1-topos, following Orton and Pitts. First, building on the cubical techniques used to give computational models of homotopy type theory, we show that there is a universe of covariant discrete fibrations, with a partial directed univalence principle asserting that functions are a retract of morphisms in this universe. To complete this retraction into an equivalence, we refine the universe of covariant fibrations using the constructive sheaf models by Coquand and Ruch.

CCS Concepts: • Theory of computation $\rightarrow$ Type theory; Constructive mathematics.

Keywords: homotopy type theory, directed type theory, directed univalence

ACM Reference Format:

Matthew Z. Weaver and Daniel R. Licata. 2020. A Constructive Model of Directed Univalence in Bicubical Sets. In *Proceedings of the 35th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS '20)*, July 8–11, 2020, Saarbrücken, Germany. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3373718.3394794>

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

LICS '20, July 8–11, 2020, Saarbrücken, Germany

© 2020 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-7104-9/20/07.

<https://doi.org/10.1145/3373718.3394794>

1 Introduction

In homotopy type theory [6, 38, 39], each type is equipped with an abstract notion of path, corresponding to the morphisms of a higher groupoid. Voevodsky’s univalence axiom states that the type of paths between types is equivalent (a generalization of isomorphic) to the type of equivalences between types. This can be seen as extending type theory with a generic program for lifting equivalences between types along any type family: given an equivalence $A \simeq B$ and any family of types $C : U \rightarrow U$, one can automatically construct an equivalence $C(A) \simeq C(B)$. Defining this lifting for all dependent types is quite subtle, but univalence has recently been given a computational interpretation via cubical type theories [2, 4, 7, 11].

A natural question is whether there are any *directed* analogues of homotopy type theory, where types are equipped with a notion of directed morphism and correspond to higher categories, generalizing groupoids. In such a setting, one possible directed analogue of univalence would circumscribe a class of type constructors that represent covariant functors, and provide a means to automatically lift a *function* $A \rightarrow B$ to a function $C(A) \rightarrow C(B)$, generalizing e.g. the Functor type class of Haskell to dependent types. By analogy with univalence, such a lifting would result from a universe classifying covariant type families that satisfies a *directed univalence* principle stating that the directed morphisms in this universe are equivalent to functions. On the mathematical side, directed type theory provides a more general language for synthetic mathematics. On the computational side, one potential application is to defining abstract syntax [16, 20, 22]; others might arise from the connections between directed homotopy theory and concurrency [15, 18, 24].

In directed type theory, a universe of covariant type families will not contain all of the usual types of type theory—e.g., Π is contravariant, not covariant, in its domain. One approach that has been taken by directed type theory is to employ some kind of modal typing discipline, tracking variances or preventing certain uses of variables [22, 24, 25], so that being a covariant family is part of the typing of a term. Another approach, recently developed by Riehl and Shulman [32], is based on instead equipping each type with *both* a notion of path and a separate notion of directed morphism (representing the arrows in a category). In this approach, all existing homotopy type theory can be interpreted using the

path structure on each type, providing a rich language of possible constructions. Being a covariant family is an *internally definable property* of a type, represented by another type that can be inhabited for a variety of type constructors—i.e. covariance is a type class. While ordinary homotopy type theory has models in *simplicial sets*, the Riehl-Shulman type theory is based on a model in *bisimplicial sets*, $\mathbf{Set}^{\Delta^{\text{op}} \times \Delta^{\text{op}}}$, where the two copies of the simplex category Δ represent these two notions of path and directed morphism. While this suffices for formalizing mathematics, for applications to computer science we would like a computational interpretation of the type theory, and a constructive account of even ordinary homotopy type theory in simplicial sets has not fully been worked out (see [17] for some recent progress).

In this paper, in place of a bisimplicial model of directed type theory, we consider a *bicubical* one, so that we can exploit the techniques that have been used to give constructive interpretations of ordinary homotopy type theory. We give a model of directed type theory with directed univalence in bicubical sets. This model is constructive in the sense of being defined in a constructive metatheory; we leave a more explicit operational semantics to future work. We have formalized¹ much of the model in Agda using the internal language approach to presheaf models of homotopy type theory [8, 23, 27, 28]. In this approach, an extensional type theory (or Agda with some axioms as a substitute [19]) is used to describe cubical-set level constructions, and the main task is to program the definitions/proofs of the *Kan operations*, which describe how to transport in each type family. Bicubical sets is also a presheaf topos, and in Section 2 we describe the axioms we add to the Agda proof assistant to work in its internal language.

In Section 3, we use this internal language to define covariant discrete fibrations (covariant functors valued in types that themselves have trivial morphism structure) and a universe classifying them. In Section 4, we show that functions are a *reflection* of morphisms in this universe: every morphism determines a function, and vice versa; converting a function to a morphism and back is the identity; but for the other round-trip there seems to be only a morphism in one direction, not a path. This reflection is most of the directed univalence *equivalence* between functions and morphisms, but to complete the equivalence, it is necessary to invert this morphism, yielding a path. To do this, Riehl and Sattler’s proof of directed univalence in the bisimplicial model [31, 35] uses a fact about the Reedy model structure on bisimplicial sets, that weak equivalences (and therefore homotopy equivalences between bifibrant objects) are *objectwise*—this means that if a morphism is an equivalence for each object of the directed category separately, then it is an equivalence. For our constructive model, we use a cubical, not simplicial, directed

category (specifically, the “Dedekind” cube category with faces, degeneracies, symmetries, diagonals, and connections, but no reversals) so that we can build universes of covariant fibrations constructively [23]. This cube category is not Reedy [34], so the proof for bisimplicial sets does not immediately apply to bicubical sets. However, for bisimplicial sets, the Reedy model structure coincides with the injective model structure (see e.g. [33, Example 7.8]), and the injective perspective applies to bicubical sets, allowing us to port this aspect of Sattler and Riehl’s proof. Recently, Shulman [37] gave a new characterization of the fibrations in the injective model structure as objectwise fibrations equipped with an algebra for the *cobar construction* (see e.g. [29, 36]). Coquand and Ruch [12, 13] give a constructive analogue of this definition as a special case of sheaf models (though the precise connection with the injective model structure remains to be worked out). In Section 5, we describe an Agda formalization of this approach, and check that the axioms describing this extension are true in bicubical sets. We then add an axiom `covEquivAx` that is true for this refined notion of fibration, and use it to complete the directed univalence equivalence for a refined covariant universe.

Related work. Our proof of the directed univalence reflection does not overlap very much with Riehl and Sattler’s directed univalence for bisimplicial sets [31, 35]—for example, we use the “glue” types used to define univalence in constructive cubical models [11], while their proof uses a dual type (called “weld” types in [26]), and we use [23] to build universes constructively. However, for completing the reflection to an equivalence, we do use a lemma from their proof, which follows from objectwise homotopy equivalences inducing homotopy equivalences.

Buchholtz and Weinberger [9] give some extensions of the Riehl-Shulman directed type theory, such as adding opposite categories, defining cartesian fibrations, and defining a universe of simplicial spaces inside of cubical spaces via a sheaf condition. In future work, we plan to investigate whether the techniques we develop here can be used to give a constructive account of directed univalence for a universe of cartesian fibrations (categories and functors, instead of groupoids and functors) as well.

Bicubical sets have also been used for type theory with parametricity: in Nuyts’s approach to parametricity for (non-homotopy) type theory [26], both cube categories are cartesian, and there is an inclusion from one interval the other. Here, we do not take an inclusion from the directed interval to the interval to be part of the cube category (but paths can be turned into morphisms in Kan types). In Cavallo and Harper’s approach to parametricity for homotopy type theory [10], the cube category used for paths is cartesian, while the cube category used for relations is semicartesian (affine logic). In contrast, our directed cube category also supports

¹See the `agda/directed` directory of <https://github.com/dlicata335/cart-cube>, starting with `LICS.agda` and `Summary.agda`

diagonals and connections to encode the simplicial shapes used by Riehl and Shulman [32].

Motivating Example. Before the technical details, we sketch a motivating example of defining abstract syntax in a directed type theory, building on [16, 20, 22]. This sketch will use a directed higher inductive type, which we do not formally study in this paper. We write $\text{Hom}_A(x, y)$ for the directed morphism type, representing a morphism from x to y —this should be thought of like the $\text{Path}_A(x, y)$ or identity $\text{Id}_A(x, y)$ type of homotopy type theory, except in general the morphisms have identity, but not necessarily composition or inverses. Those types A whose morphisms have (homotopy) unique composites (and so correspond to categories) are called *Segal* [32].

The first idea is to represent object-language contexts by a type whose elements represent contexts, and whose morphisms represent object-language substitutions or renamings. As a concrete example, we use the untyped lambda calculus, indexed by contexts denoting how many free variables are present, and take the substitutions to be weakenings only. The type of contexts should be defined as a Segal directed higher inductive type:

```
data Ctx : SegalType where
  empt : Ctx
  ext  : Ctx → Ctx
  wk   : (G : Ctx) → Hom Ctx G (ext G)
```

This declaration is intended to mean that the elements of Ctx are in bijection with the natural numbers, with directed morphisms generated by $\text{wk}_n : \text{Hom}_{\text{Ctx}}(n, n+1)$ corresponding to weakening. We declare Ctx to be Segal, which means that it should have freely generated composites; e.g. $\text{wk}_0; \text{wk}_1 : \text{Hom}_{\text{Ctx}}(0, 2)$ represents weakening the empty context with two variables. Because functions such as ext act on morphisms, we also have morphisms like $\text{ext}(\text{wk}_0) : \text{Hom}_{\text{Ctx}}(1, 2)$ that weakens in the middle of a context.

We then define a type family $\text{Var} : \text{Ctx} \rightarrow \text{UCov}$ representing the variables in each context. The type of Var expresses that, in addition to providing a type of “variables in context G for each context G ,” it takes morphisms in Ctx (i.e. weakenings) to morphisms in UCov , which by directed univalence will be functions. A particular transport function from n to $n+1$ is given for the weakening axiom—the inl injection, which is turned into a morphism in UCov using directed univalence. The elimination form for a Segal higher inductive type would then send compositions of weakenings to compositions of these injections. Thus, the type family Var includes the code for shifting a de Bruijn index.

```
Var : Ctx → UCov
Var empt    = 0
Var (ext G) = (Var G) + 1
Var (wk G)  = dua inl
```

```
data Tm (G : Ctx) : UCov where
  var : Var G → Tm G
  abs : Tm (ext G) → Tm G
  app : Tm G → Tm G → Tm G
```

Finally, we define a type family of lambda terms $\text{Tm} : \text{Ctx} \rightarrow \text{UCov}$ that is constructed from a base case Var that maps into UCov , and otherwise is a sum of products of inductive occurrences. Because UCov is closed under products, sums, and certain inductive types, Tm will *automatically* land in UCov . This means that, while the definition of Tm looks identical to what one would write in standard type theory, the usual code for lifting weakening to terms is written automatically by the transport functions for these types. We use this by

```
dtransport Tm (wk G) : Tm G → Tm (ext G)
```

to, for example, weaken a term with one extra variable, where dtransport is an operation defined for any type family that lands in UCov . General facts about directed transport will imply that these substitution actions commute with composition, which is needed in metatheory and in defining typing judgements, program logics, or dependent types.

For this example, we need a universe UCov with directed univalence, closed under products, sums, and inductive types, which is itself a category (Segal [32]); and we also need directed higher inductive types. Of these, we leave the closure of UCov under inductive types and the mechanism of directed higher inductive types to future work, and focus on defining a universe UCov supporting directed univalence.

2 Directed Type Theory in a Topos

2.1 Axioms for Cubical Type Theory

Cubical type theory [2, 4, 7, 11] extends standard dependent type theory with interval variables, which we write as $x : \mathbb{I}$. A term $x : \mathbb{I} \vdash a : A$ is a “line” in A , a term $x : \mathbb{I}, y : \mathbb{I} \vdash a : A$ is a “square”, and so on. The interval type has terms $0_{\mathbb{I}}$ and $1_{\mathbb{I}}$, substitution by which gives the endpoints of a line or the sides of a square. Dimension formulas represent subshapes of a cube—for example, the formula $x : \mathbb{I}, y : \mathbb{I} \vdash x = 0_{\mathbb{I}} \vee y = 1_{\mathbb{I}}$ represents the left and top sides of a square. The main two constructive models of homotopy type theory use the Cartesian cube category $\square_{\text{Cart}}$ [2, 4] (the free cartesian category on an interval object $\mathbb{I}$ with disjoint endpoints $0_{\mathbb{I}}$ and $1_{\mathbb{I}}$) and the de Morgan cube category [11] (which adds connection maps, giving the interval the structure of a distributive lattice, as well as reversals). A variation on the de Morgan cube category is the Dedekind cube category $\square_{\text{Ded}}$, which adds the connections but not reversals, and was shown to also give a constructive model in [28]. For the path portion of our directed homotopy type theory, we will use the axiomatic description of Cartesian cubical type theory [2], which can be interpreted in presheaves on any cartesian category with an interval object, including both the Cartesian cube category and the Dedekind cube category. To abstract the specific

choice between these, we write $\boxtimes_{\text{path}}$ to represent some cartesian cube category with an interval.

In the internal language approach [8, 23, 27, 28], models of cubical type theories are described by axioms in an extensional type theory that serves as the internal language of a presheaf 1-topos. This means that we use type theoretic syntax (formalized in Agda, with postulates for function extensionality and uniqueness of identity proofs [19]) to describe constructions in cubical sets, or, in our case, bicubical sets. In homotopy type theory based on Martin-Löf intensional type theory [38], all types are *fibrant*, which roughly means that all types have a transport function for paths. In the internal language approach, types instead denote “raw” cubical sets, which do not necessarily have such transport functions. Being fibrant is an internally definable structure on a type, in the style of type classes: for any type (family) A , there is another type classifying fibration structures on A .

We start with the axioms of Cartesian cubical type theory in [2, Section 3.1.1]. These describe the free Cartesian category generated by an interval object $\mathbb{I}$ with two distinct terms $0_{\mathbb{I}}$ and $1_{\mathbb{I}}$. A function $f : \mathbb{I} \rightarrow A$ represents a path in A , with source $f(0_{\mathbb{I}})$ and target $f(1_{\mathbb{I}})$. In cubical type theory, the

$$\begin{aligned} \text{transport} : & \quad \Pi(A : \text{Type}), (a_0, a_1 : A), (C : A \rightarrow \text{Type}). \\ & \quad \text{Path}_A(a_0, a_1) \rightarrow C(a_0) \rightarrow C(a_1) \end{aligned}$$

operation of dependent type theory is generalized to a notion of *Kan filling*, which permits the induction to go through in a recursive definition of transport for each type family C . For example, Kan filling includes a primitive notion of “filling an open box”, meaning that given three sides of a square, one can produce the missing side and an inside *filler* witnessing that the square commutes, which is used to define transport in path types. The shapes of the filling problems are determined by the *cofibrations*, which are a subset of propositions. In the internal language approach [27, 28], an Agda type iscof identifies those propositions which are cofibrations; the axioms about cofibrations needed in [2] are:

$$\begin{aligned} \text{iscof} & : \quad \Omega \rightarrow \Omega \\ \Omega_{\text{cof}} & : \quad \text{Type} \\ \Omega_{\text{cof}} & := \quad \Sigma \alpha : \Omega. \text{iscof } \alpha \\ \llbracket _ \rrbracket_{\text{cof}} & : \quad \Omega_{\text{cof}} \rightarrow \Omega \\ \llbracket \alpha \rrbracket_{\text{cof}} & := \quad \text{fst } \alpha \\ \text{iscof } \perp & : \quad \text{iscof } \perp \\ \text{iscof } \wedge & : \quad \Pi \alpha \beta : \Omega_{\text{cof}}. \text{iscof } \llbracket \alpha \rrbracket_{\text{cof}} \wedge \llbracket \beta \rrbracket_{\text{cof}} \\ \text{iscof } \vee & : \quad \Pi \alpha \beta : \Omega_{\text{cof}}. \text{iscof } \llbracket \alpha \rrbracket_{\text{cof}} \vee \llbracket \beta \rrbracket_{\text{cof}} \\ \text{iscof } \text{leq} & : \quad \Pi i j : \mathbb{I}. \text{iscof } (i = j) \\ \text{iscof } \forall_{\mathbb{I}} & : \quad \Pi \alpha : \mathbb{I} \rightarrow \Omega_{\text{cof}}. \text{iscof } (\Pi i : \mathbb{I}. \llbracket \alpha \rrbracket_{\text{cof}} i) \end{aligned}$$

This hypothesizes a predicate iscof, which determines when a proposition is a cofibration, and says that cofibrations are closed under conjunction, disjunction, equality on the interval, and universal quantification over the interval. We write Ω_{cof} for the type of cofibrations, and $\llbracket \alpha \rrbracket_{\text{cof}}$ for the underlying proposition of a cofibration. Here, Ω is the type of strict propositions in Type_0 , i.e. types such that $\Pi x, y : \alpha, x = y$.

Conjunction $\wedge$ and universal quantification Π are the Agda types, while a “squashed” disjunction $\vee$ (where any two proofs are equal) is postulated.

Given a type A and a cofibration α , a *partial term* of type A over the cofibration α is an element of $\llbracket \alpha \rrbracket_{\text{cof}} \rightarrow A$. Given a partial term $a : \llbracket \alpha \rrbracket_{\text{cof}} \rightarrow A$, the type of *extensions*² of a is defined by $A[\alpha \mapsto a] := \Sigma x : A. \Pi p : \llbracket \alpha \rrbracket_{\text{cof}}. a \, p = x$. This represents an element of A , that when α is true, restricts to a . For disjunctions, we use the notation $A[\alpha_1 \mapsto a_1, \alpha_2 \mapsto a_2]$ to denote the extension type $A[\alpha_1 \vee \alpha_2 \mapsto a]$ where a_x is the restriction of a to α_x . The disjunction $\alpha_1 \vee \alpha_2$ represents the pushout of the pullback of α_1 and α_2 , which requires that a_1 and a_2 be strictly equal along the intersection $\alpha_1 \wedge \alpha_2$. When using an extension $x : A[\alpha \mapsto a]$, we often will omit the projection from $A[\alpha \mapsto a]$ to A and write $x : A$ as well.

Using cofibrations and extensions, the *Kan operation* that fibrant types satisfy is defined as follows [2, Section 3.1.2]: Given a type Γ , a type family $A : \Gamma \rightarrow \text{Type}$ is a *Kan fibration* if one can define a relCom_{Γ} structure for it:³

$$\begin{aligned} \text{hasCom } (A : \mathbb{I} \rightarrow \text{Type}) & := \quad \Pi r r' : \mathbb{I}. \Pi \alpha : \Omega_{\text{cof}}. \\ & \quad \Pi t : (\Pi z : \mathbb{I}. \llbracket \alpha \rrbracket_{\text{cof}} \rightarrow A \, z). \\ & \quad \Pi b : (A \, r)[\alpha \mapsto t \, r]. \\ & \quad (A \, r')[\alpha \mapsto t \, r', r = r' \mapsto b] \\ \text{relCom}_{\Gamma} (A : \Gamma \rightarrow \text{Type}) & := \quad \Pi p : (\mathbb{I} \rightarrow \Gamma). \text{hasCom } (A \circ p) \end{aligned}$$

This says that a type family $A : \Gamma \rightarrow \text{Type}$ is Kan when, for any path p in Γ , any partial path t over p in A , and any total point b over the path p in A at r , we can get a new point in A over p at any location r' along the path, which restricts to t and b where they are defined.

These Kan fibrations form a model of Martin-Löf type theory with univalence and (some) higher inductive types: relCom can be inhabited for Π , Σ , path, identity, natural number, boolean, glue and some higher inductive types, and the universe itself is fibrant and univalent [2, Section 3]. We write $\mathcal{U}_{\text{Kan}} : \text{Type}$ for the universe of Kan fibrations [23]. We can define all standard concepts of homotopy type theory, and we write $\text{isEquiv } f$ and $\text{Equiv } A \, B$ for the standard definition of half-adjoint equivalence [38].

2.2 Axioms for Bicubical Type Theory

In Figure 1, we describe the additional axioms needed to describe a directed type theory based on bicubical sets. Riehl-Shulman [32] extend homotopy type theory with a directed interval $\mathbb{2}$, which is used to describe simplices (point, line, triangle, tetrahedron, etc.) by inequality formulas—e.g. the triangle is the top half of the square $x : \mathbb{2}, y : \mathbb{2} \mid y \leq x$. Here, we extend cubical type theory, which already has an interval

²This is slightly different than the extension types in [32], which combine extension in this sense with a quantifier, and are restricted in such a way that they are always fibrant.

³Formally the term after the $\mapsto$ should be a function that takes a proof of the boundary formula as input, but since we’re eliding the names for these variables, we will often write what would be in the body of that function.

$$\begin{aligned}
& 2 : \text{Type} \\
& 0_2 : 2 \\
& 1_2 : 2 \\
& 2\text{neq} : 0_2 = 1_2 \rightarrow \perp \\
& _ \sqcap _ : 2 \rightarrow 2 \rightarrow 2 \\
& \sqcap 0_2 : \Pi x : 2. x \sqcap 0_2 = 0_2 \\
& \sqcap 1_2 : \Pi x : 2. x \sqcap 1_2 = x \\
& \sqcap \text{comm} : \Pi x y : 2. x \sqcap y = y \sqcap x \\
& \sqcap \text{idem} : \Pi x : 2. x \sqcap x = x \\
& \sqcap \text{assoc} : \Pi(x, y, z : 2). \\
& \quad ((x \sqcap y) \sqcap z) = (x \sqcap (y \sqcap z)) \\
& _ \sqcup _ : 2 \rightarrow 2 \rightarrow 2 \\
& \sqcup 0_2 : \Pi x : 2. x \sqcup 0_2 = x \\
& \sqcup 1_2 : \Pi x : 2. x \sqcup 1_2 = 1_2 \\
& \sqcup \text{comm} : \Pi x y : 2. x \sqcup y = y \sqcup x \\
& \sqcup \text{idem} : \Pi x : 2. x \sqcup x = x \\
& \sqcup \text{assoc} : \Pi(x, y, z : 2). \\
& \quad ((x \sqcup y) \sqcup z) = (x \sqcup (y \sqcup z)) \\
& \text{abs}_0 : \Pi x y : 2. x \sqcap (x \sqcup y) = x \\
& \text{abs}_1 : \Pi x y : 2. x \sqcup (x \sqcap y) = x \\
& \text{isof}2\text{eq} : \Pi i j : 2. \text{isof}(i = j) \\
& \text{isof}\forall_2 : \Pi \alpha : 2 \rightarrow \Omega_{\text{cof}}. \\
& \quad \text{isof}(\Pi i : 2. \llbracket \alpha \ i \rrbracket_{\text{cof}}) \\
& 2 \text{ is tiny} \\
& \mathbb{I} \text{to}2\text{triv} : \Pi p : \mathbb{I} \rightarrow 2. \Pi x : \mathbb{I}. \\
& \quad p\ x = p\ 0_{\mathbb{I}} \\
& 2\text{mono} : \Pi p : 2 \rightarrow 2. \Pi x y : 2. \\
& \quad x = x \sqcap y \rightarrow p\ x = p(x) \sqcap p(y)
\end{aligned}$$

Figure 1. Axioms for Bicubical Directed Type Theory

for paths $\mathbb{I}$, with a second *directed interval* 2 . On the one hand, we would like the directed interval to support a constructive definition of a universe of covariant fibrations using [23], which relies on the interval being tiny (exponentiation by the interval has a right adjoint), which is not true for the interval in simplicial sets. On the other hand, we would like this interval to support the body of definitions that are made in [32], where triangular shapes are sometimes important—e.g. a type is a category (roughly) if, given any two composable directed morphisms, there is a unique composite morphism and a triangle relating them. We can accomplish both of these using the Dedekind cube category $\mathbb{C}_{\text{Ded}}$, in which the interval is defined to be a distributive lattice, where the lattice meet and join are referred to as *connections*. Using the lattice structure, we can define $x \leq y$ as $x = x \sqcap y$ (x is the min of x and y). In Figure 1, the first group of axioms describe Dedekind cubes: the interval has two disjoint points; there are meet and join operations satisfying the distributive lattice laws. Relative to the simplex category axioms [32], this omits the requirement that $\leq$ is a *total* order ($x \leq y \vee y \leq x$ for all x and y , which allows for defining a square in a type A by pasting together two triangles in A on the diagonal). This omission does not seem to be a problem: first, we have

checked that some of the definitions that use totality in [32] in fact only use it via the connections; second, many internal constructions involve Segal or discrete types, which satisfy totality weakly (with a uniqueness principle up to paths not strict equality); third, one can internally carve out the simplices via a sheaf condition [9] to state theorems about types that do satisfy totality.

The next two axioms in Figure 1, $\text{isof}2\text{eq}$ and $\text{isof}\forall_2$, say that equality on the directed interval, and Π over the directed interval, are cofibrations, extending the class of shapes that can be filled in Kan types. For example, $\text{isof}2\text{eq}$ allows converting a path to a morphism in a Kan type: given $p : \mathbb{I} \rightarrow A$, we make $2 \rightarrow A$ by the Kan composition

$$\begin{aligned}
& \text{path_to_hom}(p) := \\
& \lambda(x : 2). \text{com}_A^{z:0_{\mathbb{I}} \rightarrow 1_{\mathbb{I}}} [x = 0_2 \mapsto p(0_{\mathbb{I}}) \\
& \quad , x = 1_2 \mapsto p(1_{\mathbb{I}})](p(0_{\mathbb{I}}))
\end{aligned}$$

which has endpoints $p(0_{\mathbb{I}})$ at 0_2 and $p(1_{\mathbb{I}})$ at 1_2 . $\text{isof}\forall_2$ is used to show that glue types [11] (where the function is an equivalence) are covariant families, which is the key lemma for showing that $\mathcal{U}_{\text{cov}}$ is path-univalent. Tininess is for building $\mathcal{U}_{\text{cov}}$ [23].

The final two axioms, $\mathbb{I} \text{to}2\text{triv}$ and 2mono , are used below in the construction of directed univalence. $\mathbb{I} \text{to}2\text{triv}$ says that any map from the undirected interval to the directed interval is constant. The second axiom says that any function $2 \rightarrow 2$ is monotonic—if $x \leq y$ then $p(x) \leq p(y)$ —which, for example, rules out a reversal $1 - x$.

2.3 Soundness in Bicubical Sets

Extensional type theory with the axioms in Figure 1 can be interpreted in bicubical sets, the presheaf topos $\mathbf{Set}^{\mathbb{C}_{\text{Ded}}^{\text{op}} \times \mathbb{C}_{\text{path}}^{\text{op}}}$:

Theorem 2.1. *Let $\mathbb{C}_{\text{path}}$ be a cartesian category with an interval object $\mathbb{I}$ with two distinct endpoints $0_{\mathbb{I}}$ and $1_{\mathbb{I}}$ in $1 \rightarrow \mathbb{C}_{\text{path}} \mathbb{I}$, such that for any object C , the hom-set $C \rightarrow \mathbb{C}_{\text{path}} \mathbb{I}$ has decidable equality. Then the axioms for cartesian cubical type theory in [2, Section 3.1.1], and the axioms for bicubical directed type theory in Figure 1 are true in $\mathbf{Set}^{\mathbb{C}_{\text{Ded}}^{\text{op}} \times \mathbb{C}_{\text{path}}^{\text{op}}}$.*

The proof is in an extended version of this article [40].

3 Universe of Covariant Discrete Fibrations

Intuitively, a type $B : A \rightarrow \text{Type}$ is a covariant discrete fibration [32] when it is covariant, in the sense that given a directed morphism $\text{Hom}_A(a_0, a_1)$ there is a transport function $B(a_0) \rightarrow B(a_1)$ (satisfying a certain universal property), and each fiber $B(a)$ is discrete, in the sense that it is a groupoidal type with trivial directed morphisms. In this section, we define a universe $\mathcal{U}_{\text{cov}}$, where a type $B : A \rightarrow \mathcal{U}_{\text{cov}}$ will be a covariant discrete fibration over A . Because of the directed univalence principle we construct below, $\mathcal{U}_{\text{cov}}$ itself is a higher-categorical analogue of the category of sets and functions, but where “set” means ∞ -groupoid and function

means functor. We could similarly define a universe of contravariant discrete fibrations, but we focus (arbitrarily) on the covariant case for this paper.

Because the definition of covariant uses the directed morphism type, we first show that it follows from our axioms.

3.1 Directed Morphism Type

Given a type A and two terms a_0 and a_1 , the type of directed morphisms from a_0 to a_1 is

$$\text{Hom}_A(a_0, a_1) := \Pi i : \mathbb{2}. A[i = 0_2 \mapsto a_0, i = 1_2 \mapsto a_1]$$

Compare with the cubical path type $\text{Path}_A(a_0, a_1) := \Pi i : \mathbb{I}. A[i = 0_1 \mapsto a_0, i = 1_1 \mapsto a_1]$. We can also define dependent morphism types where, instead of a fixed type, we consider a family of types indexed by the corresponding interval type: Given a type family $A : \mathbb{2} \rightarrow \text{Type}$ and two terms $a_0 : A\ 0_2$ and $a_1 : A\ 1_2$, the type of directed paths from a_0 to a_1 is

$$\text{DHom}_A(a_0, a_1) := \Pi i : \mathbb{2}. (A\ i)[i = 0_2 \mapsto a_0, i = 1_2 \mapsto a_1]$$

In [32], morphism types are defined as extension types, which combine quantification over interval variables and extension constraints, restricted in such a way that an extension type is always fibrant. Here, we must show that the morphism type is Kan:

Theorem 3.1 (universe.Hom.Hom-code-universal). *There is a code*

$$\text{dhom} : \Pi(A : \mathbb{I} \rightarrow \text{U}_{\text{Kan}}). A(0_1) \rightarrow A(1_1) \rightarrow \text{U}_{\text{Kan}}$$

such that $\text{El}(\text{dhom}(A, a_0, a_1)) = \text{DHom}_{\text{El} \circ A}(a_0, a_1)$

The proof uses the fact that, for a directed interval variable x , $x = 0$ and $x = 1$ are cofibrations; the argument is then the same as the argument for Kan path types [2, 11].

3.2 Covariant Discrete Fibrations

For a type family $A : \mathbb{2} \rightarrow \text{Type}$ to be covariant should mean that there is a “coercion” function $A(0_2) \rightarrow A(1_2)$. For this to be well-behaved, it is necessary to moreover insist that this function has a universal property relative to A : for each input, its result is related to the input by the relation DHom_A , and in fact the unique such element of $A(1_2)$. Uniqueness here is up to homotopy (as opposed to strict equality), using the standard notion of a type A being contractible ($\text{iscontr } A := \Sigma a : A. \Pi x : A. \text{Path}_A(a, x)$)—this style of definition is one reason why the separate notions of morphisms and paths in bisimplicial setting are helpful. For a general type family $A : \Gamma \rightarrow \text{Type}$, one asks that any precomposition with a directed path in Γ has such a coercion function:

Definition 3.2 ([32, Definition 8.2]). Consider a type Γ and type family $A : \Gamma \rightarrow \text{Type}$. A is *covariant* if for every morphism $p : \text{Hom}_\Gamma(x, y)$ and point $a_0 : A\ x$, there is a unique element $a_1 : A(p\ 1_2)$ with a dependent path over $A \circ p$ starting at a_0 :

$$\begin{aligned} \text{iscov } A := & \Pi p : \mathbb{2} \rightarrow \Gamma. \Pi a_0 : A(p\ 0_2). \\ & \text{iscontr } (\sigma a_1 : A(p\ 1_2). \text{DHom}_{A \circ p}(a_0, a_1)) \end{aligned}$$

While we could use this definition unchanged, in our setting, we can give an equivalent definition that will be more convenient. First, an equivalent (for Kan types) definition of being contractible is that any partial element can be extended to a complete element [11, 27]:

$$\text{iscontr}' A := \Pi \alpha : \Omega_{\text{cof}}. \Pi p : (\llbracket \alpha \rrbracket_{\text{cof}} \rightarrow A). A[\alpha \mapsto p]$$

The idea is that taking α to be $\top$ gives the center of contraction, and taking α to be $(i = 0_1) \vee (i = 1_1)$ can be used to create a path between any two elements.

Substituting the partial-extends-to-total definition of contractibility into the definition of covariance, and rearranging some quantifiers, we obtain a cubical filling operation for morphism variables:

Definition 3.3. A *covariant filling structure* on $A : \Gamma \rightarrow \text{Type}$ is a term of type $\text{relCov}_\Gamma A$:

$$\begin{aligned} \text{hasCov } (A : \mathbb{2} \rightarrow \text{Type}) := & \Pi \alpha : \Omega_{\text{cof}}. \\ & \Pi t : (\Pi z : \mathbb{2}. \llbracket \alpha \rrbracket_{\text{cof}} \rightarrow A\ z). \\ & \Pi b : (A\ 0_2)[\alpha \mapsto t\ 0_2]. \\ & \Pi i : \mathbb{2}. (A\ i)[\alpha \mapsto t\ i, (0_2 = i) \mapsto b] \\ \text{relCov}_\Gamma (A : \Gamma \rightarrow \text{Type}) := & \Pi p : \mathbb{2} \rightarrow \Gamma. \text{hasCov}(A \circ p) \end{aligned}$$

In the formalization, we show

Theorem 3.4 (Covariant.rcov-hprop, Covariant.relCov-internal-equiv, Covariant-is-Fibrant.com-hasCov). *relCov_Γ A is a homotopy proposition, Kan, and logically equivalent (and therefore equivalent) to being covariant (Definition 3.2).*

An advantage of this formulation is that many types can be shown to be covariant by a nearly identical construction to their Kan composition structure. For example, because $\mathbb{2}$ has connections, we can apply the filling-from-composition lemma [11], which states that to construct the above, it suffices to define

$$\begin{aligned} \text{hasCov}_{\mathbb{2}}(A) := & \Pi \alpha : \Omega_{\text{cof}}. \\ & \Pi t : (\Pi z : \mathbb{2}. \llbracket \alpha \rrbracket_{\text{cof}} \rightarrow A\ z). \\ & \Pi b : (A\ 0_2)[\alpha \mapsto t\ 0_2]. \\ & (A\ 1_2)[\alpha \mapsto t\ 1_2] \end{aligned}$$

This is exactly CCHM Kan composition [11] but stated for the directed interval, rather than the path interval—thus, we can prove that the covariant types are closed under certain type constructors by the same argument used to show that they are Kan in [11].

In the formalization (Segal.agda, Discrete.agda), we also give analogous reformulations for [32]’s definitions of discrete and Segal types.

3.3 The Universe U_{cov}

We construct the universe U_{cov} following [23]: we work within Agda using an idempotent comonadic modality implemented by Andrea Vezzosi⁴ to formalize constructions involving the universe. We write U_{Kan} for the universe of Kan types constructed in [2]. We build the universe U_{cov} of

⁴This modality has been integrated into Agda as of version 2.6.1.

those Kan types that are covariant discrete fibrations (relCov) using [23, Theorem 5.2]. The assumptions of this are (1) that the interval $\mathbb{2}$ used in the definition of hasCov is *tiny*, i.e. exponentiation by the interval has a right adjoint (which we assumed in Figure 1); (2) that $\perp$ is a cofibration (an assumption from [2]); and (3) that relCov is itself Kan (Theorem 3.4); it would be immediate if we had used the original [32] definition, in terms of Π , Σ , path and hom types, all of which are Kan). Thus, we can construct a type $\mathcal{U}_{\text{cov}}$, which has decoding function $\text{El} : \mathcal{U}_{\text{cov}} \rightarrow \mathcal{U}_{\text{Kan}}$, along with an introduction rule saying that a “crisp” [23] (closed) term of type $\Gamma \rightarrow \mathcal{U}_{\text{Kan}}$ has a code in the universe for every covariant discrete fibration structure on it. Moreover, $\text{El} \circ A$ is covariant for any $A : \Gamma \rightarrow \mathcal{U}_{\text{cov}}$ —any type family in $\mathcal{U}_{\text{cov}}$ has a transport function lifting directed morphisms in its domain to morphisms in $\mathcal{U}_{\text{cov}}$ —with the definition of $\text{relCov}(\text{El} \circ A)$ determined by the data in a code. Note that the modal restrictions on universes concern its introduction rule, but the type $\mathcal{U}_{\text{cov}}$ and its elimination forms are unrestricted, and the statement of directed univalence below has no modal restrictions.

We have checked that this definition of $\mathcal{U}_{\text{cov}}$ is useful by proving it is closed under some example type constructors:

Theorem 3.5 ($\text{universe}.\{\text{Sigma}, \text{Pi}, \text{Path}, \text{Hom}\}.\text{agda}$). *There are codes*

- $\Sigma_{\text{cov}} : \Pi(A : \mathcal{U}_{\text{cov}}).\Pi(B : \text{El } A \rightarrow \mathcal{U}_{\text{cov}}).\mathcal{U}_{\text{cov}}$
- $\Pi_{\text{cov}} : \Pi(A : \mathcal{U}_{\text{Kan}}).\Pi(B : A \rightarrow \mathcal{U}_{\text{cov}}).\mathcal{U}_{\text{cov}}$
- $\text{path}_{\text{cov}} : \Pi(A : \mathcal{U}_{\text{cov}}).\text{El } A \rightarrow \text{El } A \rightarrow \mathcal{U}_{\text{cov}}$
- $\text{hom}_{\text{cov}} : \Pi(A : \mathcal{U}_{\text{cov}}).\text{El } A \rightarrow \text{El } A \rightarrow \mathcal{U}_{\text{cov}}$

that decode to Σ , Π , Path and Hom types respectively.

This shows that $\mathcal{U}_{\text{cov}}$ is closed under Σ types, Π types with a constant domain (because the domain is a contra-not covariant position; this is enforced by the modal restriction to a crisp A), Path_A , and Hom_A types. While in general $\text{Hom}_A(-, a)$ should be contravariant rather than covariant, if A is in $\mathcal{U}_{\text{cov}}$, then it is discrete, so its morphisms are equivalent to paths and thus invertible. The full proof is in the above files in the formalization; because of our reformulation of covariance as analogous to CCHM Kan filling, the proofs for Σ , Path and Hom are analogous to the arguments that Σ and Path are Kan in [11], while the argument for Π with a constant domain is analogous to the argument that Π has homogeneous composition (which does not use Kan filling in the domain).

3.4 $\mathcal{U}_{\text{cov}}$ is path-univalent

The universe $\mathcal{U}_{\text{cov}}$ should enjoy two kinds of univalence: First, like universes in ordinary homotopy type theory, a path in $\mathcal{U}_{\text{cov}}$ should be an equivalence. Second, a directed morphism should be a function. We write $\llbracket - \rrbracket_{\text{cov}}$ for the composite of the two El functions $\mathcal{U}_{\text{cov}} \rightarrow \mathcal{U}_{\text{Kan}} \rightarrow \text{Type}$. For both kinds of univalence, we use glue types [11], which are formed from a cofibration α , a partial type $T : \alpha \rightarrow \text{Type}$,

a type $B : \text{Type}$, and a partial function $f : \Pi p : \alpha.T p \rightarrow B$. The type $\text{Glue}[\alpha \mapsto (T, f)]B$ is isomorphic to $\Sigma t : (\llbracket \alpha \rrbracket_{\text{cov}} \rightarrow T).B[\alpha \mapsto f t]$, but when α is satisfied, $\text{Glue}[\alpha \mapsto (T, f)]B$ is strictly equal to T .

We consider path-univalence first, as its proof is straightforward from previous work. The main lemma is that $\mathcal{U}_{\text{cov}}$ is closed under glue types:

Lemma 3.6 ($\text{universe}.\text{Glue-Equiv-Covariant}.\text{GlueUCov}$). *Given any $\alpha : \Omega_{\text{cof}}, T : \llbracket \alpha \rrbracket_{\text{cov}} \rightarrow \mathcal{U}_{\text{cov}}$, $B : \mathcal{U}_{\text{cov}}$, and $f : \llbracket \alpha \rrbracket_{\text{cov}} \rightarrow \llbracket T \rrbracket_{\text{cov}} \rightarrow \llbracket B \rrbracket_{\text{cov}}$ where f is an equivalence, the type $\text{Glue}[\alpha \mapsto (\llbracket T \rrbracket_{\text{cov}}, f)]\llbracket B \rrbracket_{\text{cov}}$ is in $\mathcal{U}_{\text{cov}}$.*

Because of the above reformulation (relCov) of being covariant as CCHM [11] Kan filling, but for the directed interval, we can use exactly the CCHM algorithm for composition in glue types to prove this. Glue types have univalence as the special case of gluing with an equivalence on one side: $\text{Glue}[x = 0 \mapsto (A, e), (x = 1) \mapsto (B, \text{id}_B)](b)$.

Theorem 3.7 ($\text{DirUnivalence}.\text{ua}$). *For any two types A and B in $\mathcal{U}_{\text{cov}}$, we have the following equivalence.*

$$\text{ua} : \text{Equiv}(\text{Equiv} \llbracket A \rrbracket_{\text{cov}} \llbracket B \rrbracket_{\text{cov}})(\text{Path}_{\mathcal{U}_{\text{cov}}}(A, B))$$

Glue types also imply that $\mathcal{U}_{\text{cov}}$ is itself Kan—given some paths to compose in $\mathcal{U}_{\text{cov}}$, we can project paths in $\mathcal{U}_{\text{Kan}}$, which determine equivalences, which can then be composed using the glue type, which is covariant by above.

Theorem 3.8 ($\text{universe}.\text{UCov-Com}.\text{UCovU}$). *$\mathcal{U}_{\text{cov}}$ is Kan.*

The proof is analogous to the proof that $\mathcal{U}_{\text{Kan}}$ is Kan in [2, 11].

4 The Directed Univalence Retraction

Next, we define the directed univalence reflection of morphisms $\text{Hom}_{\mathcal{U}_{\text{cov}}}(A, B)$ onto functions $\llbracket A \rrbracket_{\text{cov}} \rightarrow \llbracket B \rrbracket_{\text{cov}}$.

4.1 Morphisms to Functions.

Given a morphism in $\mathcal{U}_{\text{cov}}$, part of the definition of $\mathcal{U}_{\text{cov}}$ is a covariant filling structure for $\text{El}_{\text{cov}} : \mathcal{U}_{\text{cov}} \rightarrow \text{Type}$, which yields a coercion function along from 0_2 to 1_2 :

$$\begin{aligned} \text{dcoe} &: \Pi A B : \mathcal{U}_{\text{cov}}.\text{Hom}_{\mathcal{U}_{\text{cov}}}(A, B) \rightarrow (\llbracket A \rrbracket_{\text{cov}} \rightarrow \llbracket B \rrbracket_{\text{cov}}) \\ \text{dcoe } A B p &:= \lambda x : \text{cov}_{\text{El}} p \perp \text{exfalse } x \ 1_2 \end{aligned}$$

4.2 Functions to Morphisms.

We turn a function into a morphism using the same glue type used for undirected univalence, but this time without requiring that the function f be an equivalence:

$$\begin{aligned} \text{duahom}' &: \Pi A B : \text{Type}.(A \rightarrow B) \rightarrow \text{Hom}_{\text{Type}}(A, B) \\ \text{duahom}' A B f &:= \lambda i : \mathbb{2}.\text{Glue}[i = 0_2 \mapsto (A, f), \\ &\quad i = 1_2 \mapsto (B, \text{id}_B)]B \end{aligned}$$

In presenting the definition of duahom' , we omit the boundary proofs for the sake of clarity.

For this to be a morphism in $\mathcal{U}_{\text{cov}}$, we need to show that it is Kan and covariant. In general, glue types with an arbitrary function (as opposed to an equivalence) are not Kan, but

the fact that the cofibration restricts a *directed* interval i gives some additional leverage that is not present in ordinary cubical models.

Lemma 4.1 (`universe.FunGlue.FunGlueUKan`). *For every pair of types $A B : \mathcal{U}_{\text{Kan}}$, function $f : \llbracket A \rrbracket_{\text{Kan}} \rightarrow \llbracket B \rrbracket_{\text{Kan}}$, and $i : \mathbb{2}$, the type $\text{duahom}' A B f i$ determines a code in $\mathcal{U}_{\text{Kan}}$.*

Proof. We need to show that duahom' is a Kan fibration. Thus, for any path p in $\mathbb{2} \rightarrow (\Sigma A : \mathcal{U}_{\text{Kan}}. \Sigma B : \mathcal{U}_{\text{Kan}}. A \rightarrow B)$, we construct a solution to the `hasCom` filling problem for the path in `Type` induced by the composition of p and duahom' . For the sake of clarity, we will only present the proof that this path has a coercion structure (i.e. `hasCom` from $\mathbb{0}_i$ to $\mathbb{1}_i$ over the false cofibration proposition $\perp$), but the full proof is available in the Agda formalization.

Let p_i be the path in $\mathbb{2}$ induced by p , p_A and p_B be the two paths in $\mathcal{U}_{\text{Kan}}$ induced by p , and lastly p_f be the dependent path over $\lambda j. \llbracket p_A j \rrbracket_{\text{Kan}} \rightarrow \llbracket p_B j \rrbracket_{\text{Kan}}$ contained in p . Now, given any term

$$g_0 : \text{duahom}' \llbracket p_A \mathbb{0}_i \rrbracket_{\text{Kan}} \llbracket p_B \mathbb{0}_i \rrbracket_{\text{Kan}} (p_f \mathbb{0}_i) (p_i \mathbb{0}_i)$$

we must construct a term in

$$\text{duahom}' \llbracket p_A \mathbb{1}_i \rrbracket_{\text{Kan}} \llbracket p_B \mathbb{1}_i \rrbracket_{\text{Kan}} (p_f \mathbb{1}_i) (p_i \mathbb{1}_i)$$

Note that, for any A, B, f , and i , a term in $\text{duahom}' A B f i$ is just the pair of a term $a : A$ under the assumption $i = \mathbb{0}_2$, and a term $b : B$ where $f a$ is strictly equal to b when $i = \mathbb{0}_2$. Thus, let a_0 and b_0 be the components defining g_0 . Let a_1 and b_1 denote the corresponding components we must construct to define a term in $\text{duahom}' \llbracket p_A \mathbb{1}_i \rrbracket_{\text{Kan}} \llbracket p_B \mathbb{1}_i \rrbracket_{\text{Kan}} (p_f \mathbb{1}_i) (p_i \mathbb{1}_i)$.

As p_A is a path in $\mathcal{U}_{\text{Kan}}$ and thus is Kan fibration, we can define a_1 by coercing a_0 along p_A , which we will denote as

$$a_1 := \text{com}^{\mathbb{0}_i \mapsto \mathbb{1}_i} p_A a_0$$

We must be able to define a_1 whenever $p_i \mathbb{1}_i = \mathbb{0}_2$, but it depends on a_0 which only exists when $p_i \mathbb{0}_i = \mathbb{0}_2$. Thus, for this to work, we require the triviality axiom (`lto2triv`) that states any function $\mathbb{1} \rightarrow \mathbb{2}$ is trivial. In our setting, this means that if $p_i \mathbb{1}_i = \mathbb{0}_2$ we know that for every $j : \mathbb{1}$ it must be the case that $p_i j = \mathbb{0}_2$, and therefore $p_i \mathbb{0}_i = \mathbb{0}_2$ confirming a_0 exists whenever we need to define a_1 .

Next, to define b_1 , we analogously fill along p_B , but to ensure it is strictly equal to $f \mathbb{1}_i a_1$ when $p_i \mathbb{1}_i = \mathbb{0}_2$, we must fill along the cofibration proposition $p_i \mathbb{1}_i = \mathbb{0}_2$.

$$b_1 := \text{com}^{\mathbb{0}_i \mapsto \mathbb{1}_i} p_B [p_i \mathbb{1}_i = \mathbb{0}_2 \mapsto \lambda j. p_f j (\text{com}^{\mathbb{0}_i \mapsto j} p_A a_0)] b_0$$

Again, b_1 also depends on `lto2triv` in order to define the boundary constraint.

The construction for a general composition problem in our formalization is nearly identical to that of the coercion described above. As duahom' is Kan, for every pair of types $A B : \mathcal{U}_{\text{Kan}}$, function $f : \llbracket A \rrbracket_{\text{Kan}} \rightarrow \llbracket B \rrbracket_{\text{Kan}}$, and $i : \mathbb{2}$, the type $\text{duahom}' A B f i$ determines a code in $\mathcal{U}_{\text{Kan}}$. $\square$

Lemma 4.2 (`universe.FunGlue.FunGlueUCov`). *For every pair of types $A B : \mathcal{U}_{\text{Cov}}$, function $f : \llbracket A \rrbracket_{\text{Cov}} \rightarrow \llbracket B \rrbracket_{\text{Cov}}$, and $i : \mathbb{2}$, the type $\text{duahom}' A B f i$ determines a code in $\mathcal{U}_{\text{Cov}}$.*

Proof. As we already know that duahom' is Kan, this amounts to showing it is covariant. The construction of the solution of the covariant filling problem is nearly identical to that of the Kan filling problem. The sole difference is that we use covariant filling in A and B instead of Kan filling, and we have a different justification for why a_0 always exists whenever we need to be able to define a_1 .

Let p_i be the morphism in $\mathbb{2}$ induced by the morphism we are considering in the filling problem. We must be able to define a_1 whenever $p_i \mathbb{1}_2 = \mathbb{0}_2$ using a_0 which only exists when $p_i \mathbb{0}_2 = \mathbb{0}_2$. In this case, we use the monotonicity axiom (`2mono`): if $p_i \mathbb{1}_2 = \mathbb{0}_2$, we know that for every $j : \mathbb{2}$ it also holds that $p_i j = \mathbb{0}_2$, and thus we always have a_0 when it is needed to define a_1 . $\square$

Since duahom' is both Kan and covariant, it constructs a morphism in $\mathcal{U}_{\text{Cov}}$:

$$\text{duahom} : \Pi A B : \mathcal{U}_{\text{Cov}}. (\llbracket A \rrbracket_{\text{Cov}} \rightarrow \llbracket B \rrbracket_{\text{Cov}}) \rightarrow \text{Hom}_{\mathcal{U}_{\text{Cov}}} (A, B)$$

4.3 Reflection

A calculation with the definition of Lemma 4.2 shows that

Lemma 4.3 (`DirUnivalence.duaβ`). *For every pair types $A B : \mathcal{U}_{\text{Cov}}$ and every function $f : \llbracket A \rrbracket_{\text{Cov}} \rightarrow \llbracket B \rrbracket_{\text{Cov}}$, we can construct a path*

$$\text{dua}\beta : \text{Path}_{\llbracket A \rrbracket_{\text{Cov}} \rightarrow \llbracket B \rrbracket_{\text{Cov}}} (f, \text{dcoe } A B (\text{duahom } A B f))$$

Proof. To define the path, we construct the following function of type $\mathbb{1} \rightarrow A \rightarrow B$:

$$\begin{aligned} \lambda i \ a. \text{cov}^{\mathbb{0}_2 \mapsto \mathbb{1}_2} (\text{duahom } A B f) \\ [i = \mathbb{0}_i \mapsto \lambda j. \text{glue}[j = \mathbb{0}_2 \mapsto a, j = \mathbb{1}_2 \mapsto f a] f a \\ , i = \mathbb{1}_i \mapsto \lambda j. \text{cov}^{\mathbb{0}_2 \mapsto j} (\text{duahom } A B f) a] a \end{aligned}$$

As the function has the correct boundaries at $i = \mathbb{0}_i$ and $i = \mathbb{1}_i$, it is the desired path. $\square$

However, for the other composite, we obtain only a morphism:

Lemma 4.4 (`DirUnivalence.duaηfun`). *For every $A B : \mathcal{U}_{\text{Cov}}$, $p : \text{Hom}_{\mathcal{U}_{\text{Cov}}} (A, B)$, and $i : \mathbb{2}$, we can construct a function (and therefore a $\text{Hom}_{\mathcal{U}_{\text{Cov}}}$ by Lemma 4.2)*

$$\text{dua}\eta\text{fun } i : (p \ i) \rightarrow (\text{duahom } A B (\text{dcoe } A B p) \ i)$$

Proof. The idea is that, as $\text{duahom } A B (\text{dcoe } A B p) \ i$ is a glue type with base B (i.e. $p \mathbb{1}_2$), we can take any term in $p \ i$ and coerce it to a term in $p \mathbb{1}_2$ to define our function. We construct the function as $\lambda i \ x. \text{cov}^{i \mapsto \mathbb{1}_2} p \ x$. Note that we used a different covariant filler that goes from i to $\mathbb{1}_2$; this follows from our standard covariant filler that goes from $\mathbb{0}_2$ to i using the connections on the directed interval. $\square$

5 Cobar Construction and Bicubical Stacks

Next, we will restrict $\mathcal{U}_{\text{Cov}}$ to a subuniverse for which we can show that `duaηfun` is an equivalence (we do not have countermodel showing definitively that `duaηfun` is not an

D	$: \quad U_{\text{Kan}} \rightarrow U_{\text{Kan}}$
η	$: \quad \{A : U_{\text{Kan}}\} \rightarrow \llbracket A \rrbracket_{\text{Kan}} \rightarrow \llbracket DA \rrbracket_{\text{Kan}}$
L	$: \quad \llbracket D(U_{\text{Kan}}) \rrbracket_{\text{Kan}} \rightarrow U_{\text{Kan}}$
$\tilde{D}$	$: \quad \{A : U_{\text{Kan}}\} \rightarrow (\llbracket A \rrbracket_{\text{Kan}} \rightarrow U_{\text{Kan}})$ $\rightarrow \llbracket DA \rrbracket_{\text{Kan}} \rightarrow U_{\text{Kan}}$
$\tilde{D} B$	$:= L \circ \text{Dfun}(B)$
isStack	$: \quad U_{\text{Kan}} \rightarrow U_{\text{Kan}}$
$\text{isStack } A$	$:= \text{isEquiv } \eta_A$
Dfun	$: \quad \{A B : U_{\text{Kan}}\} \rightarrow (\llbracket A \rrbracket_{\text{Kan}} \rightarrow \llbracket B \rrbracket_{\text{Kan}})$ $\rightarrow \llbracket DA \rrbracket_{\text{Kan}} \rightarrow \llbracket DB \rrbracket_{\text{Kan}}$
Dcomp	$: \quad \{A B C : U_{\text{Kan}}\} \rightarrow (g : \llbracket B \rrbracket_{\text{Kan}} \rightarrow \llbracket C \rrbracket_{\text{Kan}})$ $\rightarrow (f : \llbracket A \rrbracket_{\text{Kan}} \rightarrow \llbracket B \rrbracket_{\text{Kan}})$ $\rightarrow \text{Dfun}(g \circ f) = \text{Dfun}(g) \circ \text{Dfun}(f)$
Did	$: \quad (A : U_{\text{Kan}})$ $\rightarrow \text{Dfun}(\lambda a : \llbracket A \rrbracket_{\text{Kan}}. a) = \lambda a : \llbracket DA \rrbracket_{\text{Kan}}. a$
η_{nat}	$: \quad \{A B : U_{\text{Kan}}\} \rightarrow (f : \llbracket A \rrbracket_{\text{Kan}} \rightarrow \llbracket B \rrbracket_{\text{Kan}})$ $\rightarrow \eta_B \circ f = \eta_A \circ \text{Dfun}(f)$
η_{path}	$: \quad (A : U_{\text{Kan}})$ $\rightarrow \text{Path}_{\llbracket DA \rrbracket_{\text{Kan}} \rightarrow \llbracket D^2 A \rrbracket_{\text{Kan}}} (\text{Dfun}(\eta_A), \eta_{DA})$
Leq	$: \quad L \circ \eta_{U_{\text{Kan}}} = D$
$\text{D}\Sigma\text{snd}$	$: \quad \{A : U_{\text{Kan}}\} \rightarrow \{B : \llbracket A \rrbracket_{\text{Kan}} \rightarrow U_{\text{Kan}}\}$ $\rightarrow \Pi p : \llbracket D(\Sigma a : A.B a) \rrbracket_{\text{Kan}}. \llbracket \tilde{D} B (\text{Dfun fst } a) \rrbracket_{\text{Kan}}$
$\text{D}\Sigma\text{eq}$	$: \quad \{A : U_{\text{Kan}}\} \rightarrow (B : \llbracket A \rrbracket_{\text{Kan}} \rightarrow U_{\text{Kan}})$ $\rightarrow \text{islo } (\lambda x. (\text{Dfun fst } x, \text{D}\Sigma\text{snd } x))$

Figure 2. Axioms and Definitions for Descent Operator

equivalence for U_{cov} , but we have not been able to prove it). We do this using a notion of stack (but for the trivial topology), which picks out those types that are equivalent to their *cobar construction*. The definitions and many of the theorems in this section follow Coquand and Ruch [13]; we also check that this work applies to our setting and prove a few extra lemmas needed for our use of these ideas. Coquand and Ruch abstract many of the definitions and lemmas to the level of an arbitrary left-exact (lex) operator on types, and we have formalized this part of the construction in Agda. The parts that are cobar-specific are postulated as axioms.

We axiomatize the cobar operator as is shown in Figure 2. The main idea is that we add a new endofunctor D on fibrant types which comes equipped with a natural transformation $\eta : \text{id}_{U_{\text{Kan}}} \rightarrow D$. Using this, we make a restricted version of our covariant universe containing those covariant types A for which $\eta_A : A \rightarrow DA$ is an equivalence, and call these types stacks. We show that all of the basic type formers preserve the property of being a stack. With the added structure of this equivalence for every type, we are able to show that a function is an equivalence of bicubical stacks if at every object in $\mathfrak{D}_{\text{Ded}}$ it is an equivalence of cubical sets, from which we can complete the construction of directed univalence for the universe of covariant stacks.

5.1 Universes and Closure Properties

First, we show that the stacks are closed under the following:

Lemma 5.1 (Descent.Pi.PisStack). *Given a Kan type A and a Kan fibration $B : \llbracket A \rrbracket_{\text{Kan}} \rightarrow U_{\text{Kan}}$, if the fibers of B are stacks then $\Pi a : A.B a$ is a stack.*

Lemma 5.2 (Descent.Sigma.SisStack, Proposition 1.1 in [13]). *If a Kan type A is a stack and the fibers of a Kan fibration $B : A \rightarrow U_{\text{Kan}}$ are stacks, then $\Sigma a : A.B a$ is a stack.*

We define the universe of covariant stacks, U_{covstack} , as $\Sigma X : U_{\text{cov}}. \text{isStack } X$ ($UCovStack.UCovStack$).

Proposition 5.3. U_{covstack} is logically non-degenerate, in the sense that it contains an empty type Void .

Proof. First, Void is discrete, because being discrete means that for all $x, y : \text{Void}$, $\text{Hom}_{\text{Void}}(x, y) \simeq \text{Path}_{\text{Void}}(x, y)$ (by a particular map), which is vacuously true. Thus, $\lambda_. \text{Void}$ is a constant family of discrete types, and any such family has a code in U_{cov} . Finally, Void is a stack as well: in the specific case of cobar, $D(\text{Void}) \cong \text{Void}$, which implies the inverse to η_{Void} . Thus, Void has a code in U_{covstack} . $\square$

5.2 The Cobar Construction

Next, we work externally (so this part has not been formalized) and check that axioms in Figure 2 are true. [13] defines the cobar operator by first defining a type operator E .

Definition 5.4. Given a Kan type A in a context Γ , the type EA in context Γ has terms as follows: For every X in $\mathfrak{D}_{\text{Ded}}$ and ρ in $\Gamma(X)$, and element u in $E\Delta\rho(X)$ is given by a family of elements $u(f)$ of $A(\rho f)(Y)$ indexed by morphisms $f \in \mathfrak{D}_{\text{Ded}}(Y, X)$. It is equipped with a natural transformation $\alpha_A : A \rightarrow EA$ (fibered over each element ρ in Γ) sending a in $A\rho(X)$ to the family $f \mapsto af$.

In particular, notice that for X in $\mathfrak{D}_{\text{Ded}}$, an element of $E^n A(X)$ is a function that takes as input an n -chain of composable morphisms with codomain X and returns an element of $A(Y)$ where Y is the domain of the chain.

In the definition of the cobar operator, we will need the following: Given a list $(i_1, \dots, i_n) : \mathbb{I}^n$, the formula $\delta(i_1, \dots, i_n)$ is the $(n+1)$ -ary disjunction

$$(\mathbb{0}_{\mathbb{I}} = i_1) \vee (i_1 = i_2) \vee \dots \vee (i_{n-1} = i_n) \vee (i_n = \mathbb{1}_{\mathbb{I}})$$

We then define a partial term $\text{diag} : (\vec{i} : \mathbb{I}^n) \rightarrow \delta(\vec{i}) \rightarrow \mathbb{I}^{n-1}$ by omitting i_1 on $\mathbb{0}_{\mathbb{I}} = i_1$, i_k on $i_k = i_{k+1}$, and i_n on $i_n = \mathbb{1}_{\mathbb{I}}$.

Definition 5.5. Given a Kan type A in a context Γ , the type DA in context Γ has terms as follows: For every X in $\mathfrak{D}_{\text{Ded}}$ and ρ in $\Gamma(X)$, and element u in $D\Delta\rho(X)$ is given by a family of elements $u(i_1, \dots, i_n)$ of $E^{n+1}A$ indexed by lists of interval variables $(i_1, \dots, i_n) : \mathbb{I}^n$ for every natural number n . These families must also satisfy the following conditions:

$$\begin{array}{ll} u = \alpha \circ u \circ \text{diag} & \text{when } \mathbb{0}_{\mathbb{I}} = i_1 \\ u = E^k(\alpha) \circ u \circ \text{diag} & \text{when } i_k = i_{k+1} \\ u = E^n(\alpha) \circ u \circ \text{diag} & \text{when } i_n = \mathbb{1}_{\mathbb{I}} \end{array}$$

We define the natural transformation $\eta_A : A \rightarrow DA$ such that $(\eta_A a)(i_1, \dots, i_n) = \alpha^{n+1} a$.

While the details are rather technical, the main idea is that for every Dedekind cube X a term in $DA(X)$ is a family u_n indexed by $n : \mathbb{N}$, and each u_n is a function that takes as input an $n + 1$ -length chain of morphisms in $\mathbb{D}_{\text{Ded}}$ with codomain X , and returns an n -cube in $DA(Y)$ where Y is the domain of the chain. These families are constrained so that some of the boundaries of the $n + 1$ -cubes given by u_{n+1} are the various n -cubes given by u_n evaluated at all of the $n + 1$ -chains given by replacing an adjacent pair in the $n + 2$ -chain with their composition, along with the n -cube given by using the first morphism in the chain to substitute the n -cube given by evaluating u_n at the tail of the original chain. As a simple example, consider $f : \mathbb{D}_{\text{Ded}}(X, W)$, $g : \mathbb{D}_{\text{Ded}}(Y, X)$, and $h : \mathbb{D}_{\text{Ded}}(Z, Y)$. The square $\lambda i. j. u_2(i, j)(f, g, h)$ will have $\lambda i. u_1(i)(f, g)h$ on one side followed by $\lambda i. u_1(i)(f \circ g, h)$ on another, and $\lambda i. u_1(i)(f, g \circ h)$ is along the diagonal. $\lambda i. u_1(i)(f, g)$ is a path starting at $u_0(f)g$ and ending at $u_0(f \circ g)$, $\lambda i. u_1(i)(f \circ g, h)$ is a path from $u_0(f \circ g)h$ to $u_0(f \circ g \circ h)$, and $\lambda i. u_1(i)(f, g \circ h)$ is a path from $u_0(f)gh$ to $u_0(f \circ g \circ h)$, as depicted below:

$$\begin{array}{ccc}
 & u_0(f \circ g)h & \xrightarrow{\lambda i. u_1(i)(f \circ g, h)} u_0(f \circ g \circ h) \\
 \lambda i. u_1(i)(f, g)h \downarrow & & \nearrow \lambda i. u_1(i)(f, g \circ h) \\
 & u_0(f)gh &
 \end{array}$$

In particular, note that the conditions determine the boundaries of an n -simplex inside of every n -cube.

Proposition 5.6. *For any Kan type A , there is a path in $DA \rightarrow D^2 A$ between η_{DA} and $D\text{fun } \eta_A$.*

Proof. As is described in Proposition 4.3 of [13], an element of $D^2 A$ is of the form $v_{m,n}(i_1, \dots, i_m)(j_1, \dots, j_n)$ satisfying a number of equations. For u in DA , a homotopy from $\eta_{DA}(u)_{m,n}$ to $(D\text{fun } \eta_A)(u)_{m,n}$, in the presence of connections, is given by $\lambda k : \mathbb{I}. v_{m,n}^k$ where $v_{m,n}^k(\vec{i})(\vec{j}) := u_{m+n+1}(\vec{i} \wedge k, k \vee \vec{j})$.

Given our work is agnostic to the choice of cube category $\mathbb{D}_{\text{path}}$ for paths, we would like a proof that does not depend on the presence of connections; however, unlike in [13], we restrict our functor D to only be over fibrant types, and thus we can use path induction to replace the use of connections in their proof. Given some term $a : \mathbb{I} \rightarrow A$ for a Kan type A , we can construct a term $a_\wedge : \Pi(i, j) : \mathbb{I}^2. A[i = 0_\mathbb{I} \mapsto a \ 0_\mathbb{I}, i = 1_\mathbb{I} \mapsto a \ j, j = 0_\mathbb{I} \mapsto a \ 0_\mathbb{I}, j = 1_\mathbb{I} \mapsto a \ i, i = j \mapsto a \ i]$ by path induction on a , as the extension type is fibrant. The square for the other connection is constructed similarly. A reference implementation of the connections inside of Cartesian cubical type theory is available in the standard library of `redtt` [1, `prelude.connection`]. Having defined these, we build a homotopy with the same boundaries as $v_{m,n}^k$ by induction on m and n , iteratively using these two constructions in place of the connections.

To construct $v_{m,n}^k$ where the length of $\vec{i}$ is m and that of $\vec{j}$ is n , we first do induction over an auxiliary number $z \leq m$ to

$$\begin{aligned}
 \text{Ddua} & : \forall \{A B : \mathbb{U}_{\text{Kan}}\} \{f : \llbracket A \rrbracket_{\text{Kan}} \rightarrow \llbracket B \rrbracket_{\text{Kan}}\} \{i : 2\} \\
 & \rightarrow D(\text{duahom}' A B f i) \\
 & \rightarrow \text{duahom}' (DA) (DB) (D\text{fun } f) i \\
 \text{Ddua } g & := \text{glue}[i = 0_2 \mapsto g, i = 1_2 \mapsto D\text{fun } \text{unglue } g] \\
 & \quad (D\text{fun } \text{unglue } g) \\
 \\
 \text{Ddua-iso} & : \text{islo } _ _ \text{Ddua} \\
 \text{DPath-iso} & : \{A : \mathbb{U}_{\text{Kan}}\} \rightarrow (a_0 \ a_1 : \llbracket A \rrbracket_{\text{Kan}}) \\
 & \rightarrow \text{Iso} (D\text{Path}_{DA} \eta_{a_0} \eta_{a_1}) (D(D\text{Path}_A a_0 \ a_1)) \\
 \text{DHom-iso} & : \{A : \mathbb{U}_{\text{Kan}}\} \rightarrow (a_0 \ a_1 : \llbracket A \rrbracket_{\text{Kan}}) \\
 & \rightarrow \text{Iso} (D\text{Hom}_{DA} \eta_{a_0} \eta_{a_1}) (D(D\text{Hom}_A a_0 \ a_1)) \\
 \text{Path-}\eta\text{-eq} & : \forall \{A : \mathbb{U}_{\text{Kan}}\} \{a_0 \ a_1 : \llbracket A \rrbracket_{\text{Kan}}\} (p : D\text{Path}_A a_0 \ a_1) \\
 & \rightarrow (D\text{Path-iso } a_0 \ a_1) \circ \eta_A \circ p = \eta_{D\text{Path}_A a_0 \ a_1} p \\
 \text{Hom-}\eta\text{-eq} & : \forall \{A : \mathbb{U}_{\text{Kan}}\} \{a_0 \ a_1 : \llbracket A \rrbracket_{\text{Kan}}\} (p : D\text{Hom}_A a_0 \ a_1) \\
 & \rightarrow (D\text{Hom-iso } a_0 \ a_1) \circ \eta_A \circ p = \eta_{D\text{Hom}_A a_0 \ a_1} p
 \end{aligned}$$

Figure 3. Additional Axioms and Definitions for Cobar Operator

define a term w of type $\mathbb{I}^{z+m+n+1} \rightarrow E^{m+n+2} A$. when $z = 0$, we simply define the term to be $w_0 := \lambda \vec{x} : \mathbb{I}^{m+n+1}. u_{m+n+1}(\vec{x})$. Now, assume we have defined $w_z : \mathbb{I}^{z+m+n+1} \rightarrow E^{m+n+2} A$. To define the term $w_{z+1} : \mathbb{I}^{z+1+m+n+1} \rightarrow E^{m+n+2} A$, we isolate and curry the variable x_{z+1} from $\vec{x}$ (where x_{z+1} will ultimately correspond to the $(z + 1)$ -th element in the m -length list $\vec{i}$), and then use the $\wedge$ -connection construction on $\lambda. x_{z+1}. \lambda \vec{x} \setminus x_{z+1}. w_z(\vec{x})$, adding a new dimension variable for the connection at position x_{z+1} and resulting in w_{z+1} of the desired type. Given we can do this construction for any $z \leq m$, we in particular can do it for m itself, resulting in a term $w : \mathbb{I}^{2m+n+1} \rightarrow E^{m+n+2} A$. Lastly, we repeat the above construction starting with w instead of u , using n instead of m , using the $\vee$ -connection instead of $\wedge$, and working from the variables indexed at the end of $\vec{x}$ instead of the beginning to define a term $w' : \mathbb{I}^{2m+2n+1} \rightarrow E^{m+n+2} A$.

To finish, we abstract w' over a fresh dimension variable k and substitute back in $\vec{i}, \vec{j}$ and k in the correct positions of its arguments corresponding to the definition of $v_{m,n}^k$ that uses connections, resulting in a cube that has identical boundaries. Given this construction has the identical boundary as that built using connections, the family defined using this also satisfies the constraints required to be an element of $D^2 A$, and is the path we set out to build. $\square$

Theorem 5.7 (Corollary 4.1 in [13]). *The cobar operator satisfies the axioms in Figure 2.*

Proof. The proof that cobar operator is a lex operator (and thus satisfies the axioms) can be found in [13]. The only proof that depends on connections is that corresponding to Proposition 5.6, which we have proven above. $\square$

To complete the proofs that stacks are closed under path and morphism types, we add a few isomorphisms and equations for η to our model, shown in Figure 3.

Lemma 5.8. *For any representable $\Psi = \mathbb{I}^n \times 2^m$ and Kan fibration $A : \Psi \rightarrow \mathcal{U}_{\text{Kan}}$ in context Γ , $D(\Pi x : \Psi.A x)$ is isomorphic to $\Pi x : \Psi.D(A x)$.*

Proof. As our representables are all give by products of $\mathbb{I}$ and 2 , it is sufficient to just show it holds for each interval. First, consider $E(\Pi i : \mathbb{I}.A i)$. For each $X \in \mathcal{D}_{\text{Ded}}$, $\rho \in \Gamma(X)$, $u \in E(\Pi i : \mathbb{I}.A i)\rho$ and $f \in \mathcal{D}_{\text{Ded}}(Y, X)$, $u(f)$ is a term in $\Pi i : \mathbb{I}.(A i)\rho f$. We can reorder the arguments to get an element of $\Pi i : \mathbb{I}.E(A i)$ by $(i, X, f) \mapsto u(X, f, i)$, which is clearly an isomorphism. Similarly, for $E(\Pi i : 2.A i)$, for each $X \in \mathcal{D}_{\text{Ded}}$, $u \in E(\Pi i : 2.A i)\rho$ and $f \in \mathcal{D}_{\text{Ded}}(Y, X)$, $u(f)$ is a term in $\Pi i : 2.(A i)\rho f$. This is equivalent to considering $u(f \times \text{id}_2)$ a term in $A(Y \times 2)$, and thus we can also pull out the 2 to the outside with Yoneda and rearrange the arguments to get a term in $\Pi i : 2.E(A i)$. Given how DA is constructed using EA , we can iterate this isomorphism and commute arguments with the indexing lists of interval variables to get the isomorphisms between $D(\Pi i : \mathbb{I}.A i)$ and $\Pi i : \mathbb{I}.D(A i)$, along with between $D(\Pi i : 2.A i)$ and $\Pi i : 2.D(A i)$. As this is simply reordering the arguments to the families, the equations required by D still hold. $\square$

Lemma 5.9. *For any representable $\Psi = \mathbb{I}^n \times 2^m$, Kan fibration $A : \Psi \rightarrow \mathcal{U}_{\text{Kan}}$ in context Γ , cofibration β only depending on Ψ , and partial term $a : \Pi x : \Psi.\beta \rightarrow A x$, the type $D(\Pi x : \Psi.(A x)[\beta \mapsto a x])$ is isomorphic to $\Pi x : \Psi.(D(A x))[\beta \mapsto \eta_{A x} a x]$.*

Proof. Given how $D(A x)$ is defined using E , we can unfold the definition and use the above isomorphism to see that $D(\Pi x : \Psi.(A x)[\beta \mapsto a x])$ is a restriction of the type $\Pi x : \Psi.D(A x)$ given by $\Pi x : \Psi.(D(A x))[\beta \mapsto \eta_{A x} a x]$. $\square$

Corollary 5.10. *The axioms $D\text{Path-iso}$, $D\text{Hom-iso}$, $\text{Path-}\eta\text{-eq}$ and $\text{Hom-}\eta\text{-eq}$ from Figure 3 hold.*

Lemma 5.11 (`Descent.Path.Path0-isStack`). *If the fibers of a Kan fibration $A : \mathbb{I} \rightarrow \mathcal{U}_{\text{Kan}}$ are stacks, then for any terms a_0 in $A \mathbb{0}_\mathbb{I}$ and a_1 in $A \mathbb{1}_\mathbb{I}$ the type $D\text{Path}_A a_0 a_1$ is a stack.*

Proof. To construct the inverse to η , we use the isomorphism from Corollary 5.10 and construct an inverse to the function $\eta_A \circ - : D\text{Path}_A a_0 a_1 \rightarrow D\text{Path}_{D \circ A} (\eta_A \mathbb{0}_\mathbb{I} a_0) (\eta_A \mathbb{1}_\mathbb{I} a_1)$. Given a path p in $D\text{Path}_{D \circ A} (\eta_A \mathbb{0}_\mathbb{I} a_0) (\eta_A \mathbb{1}_\mathbb{I} a_1)$ and i in 2 , the inverse just applies $\eta_{A i}^{-1}$ to $p i$, using the fact that $A i$ is a stack to compose with the path in $\text{Path}_{A i} (\eta_{A i}^{-1} (\eta (p i))) (p i)$ when i equals $\mathbb{0}_\mathbb{I}$ or $\mathbb{1}_\mathbb{I}$. As $(\eta_{A i}, \eta_{A i}^{-1})$ form an adjoint equivalence for every i , it follows that this is also an adjoint equivalence and thus an equivalence. $\square$

Analogously, we have

Lemma 5.12 (`Descent.Hom.Hom0-isStack`). *If the fibers of a Kan fibration $A : 2 \rightarrow \mathcal{U}_{\text{Kan}}$ are stacks, then for any terms a_0 in $A \mathbb{0}_2$ and a_1 in $A \mathbb{1}_2$ the type $D\text{Hom}_A a_0 a_1$ is a stack.*

As with path and hom types, we add an isomorphism as an axiom in our model to facilitate the proof that stacks are closed under duahom' .

Lemma 5.13. *Consider Kan types A and B , function $f : \llbracket A \rrbracket_{\text{Kan}} \rightarrow \llbracket B \rrbracket_{\text{Kan}}$ and $i : 2$. The function $D\text{dua}$ defined in Figure 3 is an isomorphism.*

Proof. Observe that, for every object $\Psi = \mathbb{I}^m \times 2^n$, the type $\Psi \rightarrow \text{duahom}' A B f i$ is isomorphic to $\text{duahom}' (\Psi \rightarrow A) (\Psi \rightarrow B) (f \circ -) i$. Unfolding the definition of D , we see the action of $D\text{dua}$ is an application of this isomorphism, and thus is an isomorphism itself. $\square$

Lemma 5.14 (`Descent.FunGlue.Glue-isStack`). *Given Kan types A and B , a function $f : \llbracket A \rrbracket_{\text{Kan}} \rightarrow \llbracket B \rrbracket_{\text{Kan}}$ and a term $x : 2$, if A and B are stacks, then so is the type $\text{duahom}' A B f i$.*

We use the following property below:

Theorem 5.15 (Theorem 5.1 in [13]). *Given stacks A and B in $\mathcal{U}_{\text{Kan}}$ and a function $f : \llbracket A \rrbracket_{\text{Kan}} \rightarrow \llbracket B \rrbracket_{\text{Kan}}$, if for every $X \in \mathcal{D}_{\text{Ded}}$ $f(-, X)$ is an equivalence of cubical sets, then f is an equivalence of bicubical sets.*

5.3 Completing Directed Univalence

Next, we show that $\mathcal{U}_{\text{covstack}}$ satisfies directed univalence.

First, as $\mathcal{U}_{\text{covstack}}$ is the restriction of $\mathcal{U}_{\text{cov}}$ by a homotopy proposition isStack , the proof of the retraction in $\mathcal{U}_{\text{cov}}$ lifts into $\mathcal{U}_{\text{covstack}}$ (see `DirUnivalenceReflectionStack.agda`). To complete the equivalence, it suffices to show that duanfun is an equivalence; then, by function extensionality and path-univalence for $\mathcal{U}_{\text{covstack}}$, we get a path in $2 \rightarrow \mathcal{U}_{\text{covstack}}$. To construct this, we will externally justify the following axiom:

Definition 5.16 (Covariant Equivalence Axiom).

$$\begin{aligned} \text{covEquivAx} : & \Pi p q : 2 \rightarrow \mathcal{U}_{\text{covstack}}. \\ & \Pi f : (\Pi i : 2. \llbracket p i \rrbracket_{\text{covstack}} \rightarrow \llbracket q i \rrbracket_{\text{covstack}}). \\ & \Pi e_0 : \text{isEquiv}(f \mathbb{0}_2). \Pi e_1 : \text{isEquiv}(f \mathbb{1}_2). \\ & \Pi i : 2. \text{isEquiv}(f i) \end{aligned}$$

which says that any map between covariant fibrations that is an equivalence at $\mathbb{0}_2$ and at $\mathbb{1}_2$ is an equivalence. Before justifying the axiom, we use it to complete the internal directed univalence equivalence.

Lemma 5.17 (`DirUnivalence.duan`). *For every pair of types $A B : \mathcal{U}_{\text{covstack}}$ and every morphism $p : \text{Hom}_{\mathcal{U}_{\text{covstack}}}(A, B)$, the map duanfun defined in Lemma 4.4 is an equivalence, so by path-univalence for $\mathcal{U}_{\text{covstack}}$ we obtain*

$$\text{duan} : \text{Path}_{\text{Hom}_{\mathcal{U}_{\text{covstack}}}(A, B)}(p, \text{duahom } A B (\text{dcoe } A B p))$$

Proof. To build this path, we use covEquivAx with duanfun , as on both $\mathbb{0}_2$ and $\mathbb{1}_2$ duanfun is strictly equal to the identity function and thus is an equivalence. We will denote the proofs that it is an equivalence on the endpoints by e_0 and e_1 respectively. As the paths at the endpoints are induced by the identity functions, we can also construct paths between the the paths induced by duanfun at $\mathbb{0}_2$ and $\mathbb{1}_2$ and the identity

paths at A and B , which we will denote by p_0 and p_1 . We now construct the path $\text{dua}\eta$ as follows:

$$\begin{aligned} \lambda i j. (\text{com}^{0_1 \mapsto 1_1} (\lambda _ . \text{Path}_{\text{U}_{\text{covstack}}} (p j, \text{duahom } A B (\text{dcoe } A B p) j)) \\ [j = 0_2 \mapsto \lambda i. p_0 i, \\ j = 1_2 \mapsto \lambda i. p_1 i] \\ \text{ua } (\text{covEquivAx } _ _ \text{dua}\eta \text{fun } e_0 e_1 j)) i \end{aligned}$$

□

Theorem 5.18 (*DirUnivalence.dua*). *For all $A B : \text{U}_{\text{covstack}}$, we have an equivalence*

$$\text{dua} : \text{Equiv} (\llbracket A \rrbracket_{\text{covstack}} \rightarrow \llbracket B \rrbracket_{\text{covstack}}) (\text{Hom}_{\text{U}_{\text{covstack}}} (A, B))$$

Proof. As we have functions between $\llbracket A \rrbracket_{\text{covstack}} \rightarrow \llbracket B \rrbracket_{\text{covstack}}$ and $\text{Hom}_{\text{U}_{\text{covstack}}} (A, B)$ that are inverses up to homotopy, we have a quasi-equivalence and thus an equivalence. □

Putting all of these results together, we conclude:

Theorem 5.19 (Main Theorem). *There exists a constructive model of a type theory in bicubical sets with a universe of fibrant types (U_{Kan}) and a universe of covariant fibrations ($\text{U}_{\text{covstack}}$) such that:*

- $\text{U}_{\text{covstack}}$ has a decode function into U_{Kan} ;
- U_{Kan} is closed under Π , Σ , DPath , DHom and contains codes for (smaller) U_{Kan} and $\text{U}_{\text{covstack}}$;
- $\text{U}_{\text{covstack}}$ is closed under Π (with a fixed closed domain), Σ , DPath and DHom ;
- U_{Kan} and $\text{U}_{\text{covstack}}$ are both path univalent;
- $\text{U}_{\text{covstack}}$ is morphism (directed path) univalent.

The model described by Theorem 5.19 could be presented in such a way that the user only sees fibrant (Kan) types, with a homotopy type theory living in U_{Kan} , and a fixed collection of covariant type constructors living in $\text{U}_{\text{covstack}}$. That is, the type constructors that land in $\text{U}_{\text{covstack}}$ are special in two ways relative to the general universe U_{Kan} that they are drawn from—they are covariant families, and they are families of stacks. The reason we do not restrict attention to a universe $\text{U}_{\text{KanStack}}$ of Kan stacks, is that at present we only know that $\text{U}_{\text{covstack}}$ itself is in U_{Kan} . In our current approach, to make $\text{U}_{\text{covstack}}$ internally extensible (by proving an internal covariance predicate as in Definitions 3.2/3.3), we would also need to expose the isStack predicate and its definition or closure conditions, so that the user can also prove that a type family is a family of stacks. If we could show that $\text{U}_{\text{covstack}}$ is itself a stack, then we could hide all of the stacks from the user by interpreting the homotopy type theory in $\text{U}_{\text{KanStack}}$.

5.4 The Equivalence Axiom in Bicubical Sets

Having used the equivalence axiom to complete our main theorem, we now provide an external constructive proof justifying its inclusion in the model. While we do not define a model structure on bicubical sets, we do use some aspects of the type-theoretic model structure on cubical sets (see [5, 14]) in the following definitions and proofs.

Definition 5.20. A morphism $f \in \text{Set}^{\text{op}_{\text{path}} \times \text{op}_{\text{Ded}}}$ is a *left map* if, for every $n \in \mathbb{N}$, the commuting square of cubical sets shown below is a homotopy pullback.

$$\begin{array}{ccc} A(_, n) & \xrightarrow{0_2} & A(_, 0) \\ f(_, n) \downarrow & & \downarrow f(_, 0) \\ \Gamma(_, n) & \xrightarrow{0_2} & \Gamma(_, 0) \end{array}$$

The map denoted by 0_2 is that induced by the substitution sending all directed interval variables to 0_2 . A morphism is a *left fibration* if it is both a left map and a Kan fibration.

This definition unfolds to the equivalent statement that, for any directed n -cube in Γ and any point in A over its zero vertex, one can construct a unique (up to homotopy) directed n -cube in A over that in Γ .

The internal definition of relCov for a type family $A : \Gamma \rightarrow \text{Type}$ picks out the same morphisms of bicubical sets.

Lemma 5.21. *Given some internal type family $B : A \rightarrow \text{U}_{\text{cov}}$ in an open context Γ (i.e. $\Gamma \vdash B : A \rightarrow \text{U}_{\text{cov}}$), the corresponding external morphism $\Sigma x : \Gamma. \Sigma y : A x. B x y \rightarrow \Sigma x : \Gamma. A x$ is a left map.*

Proof. As described in Definition 3.3, the definition of relCov picks out type families for which, given any morphism p in Γ and term in A over p at 0_2 , there is a unique morphism in A over p ; however, as any instance of relCov can be used in an open context and the category $\mathbb{B}_{\text{Ded}}$ is the free Cartesian category generated from $\mathbb{2}$, the instance can be used n many times in a context with (at least) n many directed interval variables to construct the solution for the directed n -cube. Thus, the external morphism is a left map. □

The covariant equivalence axiom is a direct corollary of a general lemma from [35]. As with the definition of left map, the lemma originally was stated for the Reedy model structure on bisimplicial sets. The proof itself only uses the definition of left map as a level wise homotopy pullback over the zero cells (Lemma 5.21), and the fact that weak equivalences are level wise weak equivalences (Theorem 5.15); thus, it also holds in our setting. The lemma is as follows.

Lemma 5.22. *Given bicubical sets Γ, A and B , two left fibrations $f : A \rightarrow \Gamma$ and $g : B \rightarrow \Gamma$, and a morphism $e : A \rightarrow B$ such that $f = g \circ e$, the morphism e is a weak equivalence of bicubical sets when $e(_, 0) : A(_, 0) \rightarrow B(_, 0)$ is a weak equivalence of cubical sets.*

Proof. For each $n \in \mathbb{N}$ consider the following diagram.

$$\begin{array}{ccccc} A(_, n) & \xrightarrow{0_2} & A(_, 0) & & \\ \downarrow f(_, n) & \searrow e(_, n) & \downarrow e(_, 0) & & \\ & B(_, n) & \xrightarrow{0_2} & B(_, 0) & \\ & \downarrow f(_, 0) & & \downarrow g(_, 0) & \\ \Gamma(_, n) & \xrightarrow{0_2} & \Gamma(_, 0) & & \\ & \downarrow g(_, n) & & \downarrow g(_, 0) & \\ & \Gamma(_, n) & \xrightarrow{0_2} & \Gamma(_, 0) & \end{array}$$

As both the front and back squares are homotopy pullback diagrams and $e(_, 0)$ is a weak equivalence of cubical sets, it follows that $e(_, n)$ is also a weak equivalence of cubical sets. As weak equivalences are level wise weak equivalences of cubical sets, e is a weak equivalence of bicubical sets. $\square$

Corollary 5.23. *Consider bicubical sets Γ , A and B , two left fibrations $f : A \rightarrow \Gamma \times 2$ and $g : B \rightarrow \Gamma \times 2$, and a morphism $e : A \rightarrow B$ such that $(f, p) = (g, q) \circ e$. Let $A_{2(0)}$ denote the homotopy pullback of $f(_, 0)$ and the 0_2 -projection $\Gamma \times 2(_, 0) \hookrightarrow \Gamma(_, 0) \times 2(_, 0)$ (and define $B_{2(0)}$ analogously) as shown in the following diagram.*

$$\begin{array}{ccc} A_{2(0)} & \xrightarrow{0_2} & A(_, 0) \\ f_{2(0)} \downarrow & & \downarrow f(_, 0) \\ \Gamma \times 2(_, 0) & \xrightarrow{0_2} & \Gamma(_, 0) \times 2(_, 0) \end{array}$$

Let $e_{2(0)} : A_{2(0)} \rightarrow B_{2(0)}$ be the morphism induced by e . If $e_{2(0)}$ is a weak equivalence of bicubical sets, then e is also a weak equivalence of bicubical sets.

Proof. As $e_{2(0)}$ is a weak equivalence of bicubical sets, it is a level wise weak equivalence of cubical sets. In particular, $e_{2(0)}(_, 0) : A_{2(0)}(_, 0) \rightarrow B_{2(0)}(_, 0)$ is a weak equivalence. Observe that $A_{2(0)}(_, 0) = A(_, 0)$ (and similarly for B), and $e_{2(0)}(_, 0) = e(_, 0)$. As $e(_, 0)$ is a weak equivalence of cubical sets, Lemma 5.22 allows us to conclude that e is a weak equivalence of bicubical sets. $\square$

Theorem 5.24. *covEquivAx holds in our model.*

Proof. In order to see Corollary 5.23 as a justification of the covariant equivalence axiom (Definition 5.16), we must make a couple observations. First, from Lemma 5.21, we know that the internal term $\Gamma \vdash p : 2 \rightarrow \mathcal{U}_{\text{cov}}$ corresponds to the external left fibration $\Sigma x : \Gamma. \Sigma i : 2. p \ x \ i \rightarrow \Gamma \times 2$ (and analogously for q). Thus, in the external corollary, we can instantiate A and B with the external closed bicubical sets $\Sigma x : \Gamma. \Sigma i : 2. p \ x \ i$ and $\Sigma x : \Gamma. \Sigma i : 2. q \ x \ i$. Let A and B denote these bicubical sets in the following proof. The second item of note is that the pullback defining $A_{2(0)}$ in the statement of the corollary corresponds to the restriction of $A \rightarrow \Gamma \times 2$ to the components over the two endpoints 0_2 and 1_2 of 2 . Thus, the internal proofs e_0 and e_1 in the open context Γ demonstrating f is an equivalence over the endpoints of the directed interval matches up with the equivalence needed in the statement of Corollary 5.23.

We also need to confirm that the proof of covEquivAx is stable under substitution, and thus can be an internal axiom. Given covEquivAx uses Theorem 5.15 to construct the equivalence from the input data, we first show that for any A , B and $f : A \rightarrow B$ where f is a point-wise equivalence, the term of $\text{isEquiv } f$ built using Theorem 5.15 is stable under (strict) pullbacks. As the construction in Theorem 5.15 described in Theorem 5.1 of [13] is computed point-wise with respect to

$\mathcal{D}_{\text{Ded}}$ and pullbacks of bicubical sets are also point-wise pullbacks of cubical sets, the pullback commutes with the construction and thus Theorem 5.15 commutes with substitution. The second construction that must be stable under pullbacks is the level-wise weak equivalence defined in Lemma 5.22. In particular, given cubical sets Γ , A and B , left fibrations $f : A \rightarrow \Gamma$ and $g : B \rightarrow \Gamma$, a morphism $e : A \rightarrow B$ such that $f = g \circ e$, and a proof $v_0 : \text{isEquiv } e(_, 0)$, we need to know that every $v_n : \text{isEquiv } e(_, n)$ created by the proof is stable under pullback. Again, as each v_n is computed separately for each object in $\mathcal{D}_{\text{Ded}}$, the pullback commutes with the point-wise argument, and as every v_n is constructed as the solution of a covariant filling problem (which in particular commutes with pullbacks), the construction in Lemma 5.22 is also stable under substitution. $\square$

6 Future Work

We expect that the internal language axioms in Section 2 can be given an operational semantics following [3, 21]: the new axioms $\mathbb{I} \rightarrow 2 \text{triv}$ and 2mono are used only to refute certain possibilities in showing that the definitions of Lemmas 4.1 and 4.2 type check, and the cobar construction used to complete directed univalence is entirely constructive. We also are interested in developing the constructive theory of inductive and higher inductive types in a directed setting, opening up the potential for directed type theory to be a useful setting for the formal verification of computational structures.

For the use of our constructive model for formalization of higher category theory, we conjecture that our type theory on bicubical sets can support a model of $(\infty, 1)$ -categories in the same way that the Segal/Rezk types [32] in bisimplicial sets do. First, we have observed that the bicubical sets satisfying the Segal or Rezk condition also weakly satisfy a sheaf condition on the directed cubes that makes them equivalent to simplicial-cubical sets. Thus, if we use the equivariant Cartesian cubical sets model proposed by Awodey, Cavallo, Coquand, Riehl and Sattler as the base cubical type theory—which is Quillen equivalent to the simplicial model [30]—it seems likely that in the resulting model Segal/Rezk bicubical sets would be equivalent to Segal/Rezk bisimplicial sets.

Acknowledgments

This material is based upon work supported by the Air Force Office of Scientific Research under Grant No. FA9550-17-1-0363, No. FA9550-15-1-0053, and No. FA9550-16-1-0292. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author and do not necessarily reflect the views of the Air Force Office of Scientific Research. We thank Evan Cavallo, Thierry Coquand, Emily Riehl, Mitchell Riley, Christian Sattler, Mike Shulman, Jonathan Weinberger and the anonymous reviewers for their helpful insights and feedback.

References

- [1] [n.d.]. reddit. <https://github.com/RedPRL/reddit>.
- [2] Carlo Angiuli, Guillaume Brunerie, Thierry Coquand, Kuen-Bang Hou (Favonia), Robert Harper, and Daniel R. Licata. 2019. Syntax and Models of Cartesian Cubical Type Theory. (2019). Available from <https://github.com/dlicata335/cart-cube/blob/master/cart-cube.pdf>.
- [3] Carlo Angiuli, Kuen-Bang Hou (Favonia), and Robert Harper. 2017. Computational Higher Type Theory III: Univalent Universes and Exact Equality. (2017). arXiv:1712.01800.
- [4] Carlo Angiuli, Kuen-Bang Hou (Favonia), and Robert Harper. 2018. Cartesian Cubical Computational Type Theory: Constructive Reasoning with Paths and Equalities.. In *Computer Science Logic*.
- [5] Steve Awodey. 2019. A Quillen model structure on the category of cartesian cubical sets. (2019). Talk at *Homotopy Type Theory 2019*, available from <https://hott.github.io/HoTT-2019>.
- [6] S. Awodey and M. Warren. 2009. Homotopy theoretic models of identity types. *Mathematical Proceedings of the Cambridge Philosophical Society* (2009).
- [7] Marc Bezem, Thierry Coquand, and Simon Huber. 2013. A model of type theory in cubical sets. (September 2013). Preprint.
- [8] Lars Birkedal, Ales Bizjak, Ranald Clouston, Hans Bugge Grathwohl, Bas Spitters, and Andrea Vezzosi. 2016. Guarded Cubical Type Theory: Path Equality for Guarded Recursion. (2016). arXiv:1606.05223.
- [9] Ulrik Buchholtz and Jonathan Weinberger. 2019. Type-theoretic modalities for synthetic $(\infty, 1)$ -categories. (2019). Talk at Homotopy Type Theory 2019.
- [10] Evan Cavallo and Robert Harper. 2019. Parametric Cubical Type Theory. (2019). arXiv:1901.00489.
- [11] Cyril Cohen, Thierry Coquand, Simon Huber, and Anders Mörtberg. 2018. Cubical Type Theory: a constructive interpretation of the univalence axiom. In *Post-proceedings of the 21st International Conference on Types for Proofs and Programs (TYPES 2015)*, T. Uustalu (Ed.), 5:1–5:34. doi: 10.4230/LIPIcs.TYPES.2015.5.
- [12] Thierry Coquand. 2019. Constructive Presheaf Models of Univalence. (2019). Talk at Homotopy Type Theory 2019.
- [13] Thierry Coquand and Fabian Ruch. 2019. Constructive sheaf models of type theory. (2019). arXiv:1912.10407.
- [14] Thierry Coquand and Christian Sattler. 2016. A model structure on some presheaf categories. (2016). Available from cse.chalmers.se/~coquand/mod2.pdf.
- [15] Lisbeth Fajstrup, Eric Goubault, Emmanuel Haucourt, Samuel Mimram, and Martin Raussen. 2016. *Directed algebraic topology and concurrency*. Springer.
- [16] Marcelo Fiore, Gordon Plotkin, and Daniele Turi. 1999. Abstract Syntax and Variable Binding. In *IEEE Symposium on Logic in Computer Science*.
- [17] Nicola Gambino and Simon Henry. 2019. Towards a constructive simplicial model of Univalent Foundations. (2019). <https://arxiv.org/abs/1905.06281>.
- [18] M. Grandis. 2009. *Directed Algebraic Topology: Models of non-reversible worlds*. Cambridge University Press.
- [19] Martin Hofmann. 1995. *Extensional Concepts in Intensional Type Theory*. Ph.D. Dissertation. University of Edinburgh.
- [20] Martin Hofmann. 1999. Semantical analysis of higher-order abstract syntax. In *IEEE Symposium on Logic in Computer Science*.
- [21] Simon Huber. 2018. Canonicity for Cubical Type Theory. *Journal of Automated Reasoning* (2018). <https://doi.org/10.1007/s10817-018-9469-1>
- [22] Daniel R. Licata and Robert Harper. 2011. 2-Dimensional Directed Type Theory. In *Mathematical Foundations of Programming Semantics (MFPS)*.
- [23] Daniel R. Licata, Ian Orton, Andrew M. Pitts, and Bas Spitters. 2018. Internal Universes in Models of Homotopy Type Theory. In *International Conference on Formal Structures for Computation and Deduction*.
- [24] Paige Randall North. 2018. Towards a directed homotopy type theory. (2018). arXiv:1807.10566.
- [25] Andreas Nuyts. 2015. *Towards a directed HoTT based on 4 kinds of variance*. Master's thesis. KU Leuven.
- [26] Andreas Nuyts. 2017. A Model of Parametric Dependent Type Theory in Bridge/Path Cubical Sets. (2017). arXiv:1706.04383.
- [27] Ian Orton and Andrew M. Pitts. 2016. Axioms for Modelling Cubical Type Theory in a Topos. In *Computer Science Logic*.
- [28] I. Orton and A. M. Pitts. 2018. Axioms for Modelling Cubical Type Theory in a Topos. *Logical Methods in Computer Science* 14, 4:23 (Dec. 2018), 1–33. Special issue for CSL 2016.
- [29] Emily Riehl. 2014. *Categorical Homotopy Theory*. Cambridge University Press.
- [30] Emily Riehl. 2019. The equivariant uniform kan fibration model of cubical homotopy type theory. (2019). Talk at Homotopy Type Theory 2019.
- [31] Emily Riehl, Evan Cavallo, and Christian Sattler. 2018. On the directed univalence axiom. Talk at the AMS Special Session on Homotopy Type Theory, Joint Mathematics Meetings.
- [32] Emily Riehl and Michael Shulman. 2018. A type theory for synthetic ∞ -categories. *Higher Structures* 1, 1 (2018).
- [33] Emily Riehl and Dominic Verity. 2014. The theory and practice of Reedy categories. *Theory and Applications of Categories* 29, 9 (2014).
- [34] Christian Sattler. 2018. Do cubical models of type theory also model homotopy types? (2018). Talk at Workshop on Types, Homotopy Type theory, and Verification, Hausdorff Institute for Mathematics.
- [35] Christian Sattler and Emily Riehl. 2018. Directed univalence for the left fibration classifier. (2018). Private correspondence.
- [36] Michael Shulman. 2009. Homotopy limits and colimits and enriched homotopy theory. (2009). <https://arxiv.org/abs/math/0610194>.
- [37] Michael Shulman. 2019. All $(\infty, 1)$ -toposes have strict univalent universes. (2019). arXiv:1904.07004.
- [38] The Univalent Foundations Program, Institute for Advanced Study. 2013. *Homotopy Type Theory: Univalent Foundations Of Mathematics*. Available from homotopytypetheory.org/book.
- [39] Vladimir Voevodsky. 2006. A very short note on homotopy λ -calculus. *Unpublished* (September 2006), 1–7. http://www.math.ias.edu/vladimir/files/2006_09_Hlambda.pdf
- [40] Matthew Z. Weaver and Daniel R. Licata. 2020. A Constructive Model of Directed Univalence in Bicubical Sets (extended version). (2020). Available from <https://dlicata.wescreates.wesleyan.edu/>.